

TrustUs

Server — Consolidated Fix & Feature Plan

Every confirmed gap, in one place, before any code is written

Everything here is in scope — no priority order implied. Build reference only.

Table of Contents

0. Purpose

Every gap found across the audit, the Server Production Reference, the Web Dashboard doc, and the direct code/regulation checks in this conversation is consolidated here into one build-ready reference. Nothing is sequenced into phases — all of it is in scope, per your instruction to fix everything rather than pick a subset first. Sections are grouped by theme purely for readability while writing code, not by priority.

1. New Feature: Flexible Regulatory Disclosure System

This replaces the idea of hardcoding pharma's 9 Schedule H2 fields as fixed columns. Instead, any category (pharma, auto parts, FMCG, electronics, or one not yet imagined) can define its own required disclosure fields, and any product in that category collects values for them — with zero schema migrations needed when a new category or a new regulation shows up. The QR code itself never changes — it still encodes only a plain URL. This section only adds richer data behind that URL.

1.1 Why This Approach, Specifically

- Schedule H2 already proved the pharma-hardcoded approach would need a schema change the moment a second regulation or a second country's rules applied — the 2026 amendment (GSR 506(E)) itself expanded scope from 300 brands to all vaccines/antimicrobials/anticancer/NDPS drugs, showing these requirements change over time even within one country.
- A template-driven system means adding a new required field to an existing category, or defining an entirely new category's requirements, is a data entry action by a platform admin — not a code deployment.

1.2 Schema

Table	Key Columns	Purpose
disclosure_field_definitions	field_id PK, category (e.g. 'pharma', 'auto_parts'), field_key (e.g. 'generic_name'), label, field_type (text date multiline list), required (bool), display_order	The template: what fields this category must collect. Seeded once for 'pharma' with the 9 Schedule H2 elements; empty for other categories until someone defines them.
product_disclosures	product_id FK, field_id FK, value (text), PRIMARY KEY(product_id, field_id)	The actual values a manufacturer entered for their product, against whichever template applies to that product's category.

1.3 Seed Data for Pharma (Schedule H2, Rule 96(6)-(7))

These 9 rows are inserted once as disclosure_field_definitions where category='pharma' — confirmed against the actual regulation text, not assumed:

- unique_product_id (serial number for that pack — this can simply reference the platform's own unit_id, no separate field needed if reused directly)
- generic_name (proper/generic name of the drug)
- brand_name
- manufacturer_name_address
- batch_number (already exists as a first-class Batch field — map to it rather than duplicating)
- mfg_date (already exists as a first-class Batch field — map to it)

- expiry_date (already exists as a first-class Batch field — map to it)
- manufacturing_licence_number
- excipients_qualitative (optional — required=false, since the regulation says “where applicable”)

Three of the nine (batch number, mfg date, expiry date) already exist as real Batch columns — the disclosure system should reference those directly rather than duplicating them, to avoid two sources of truth for the same fact.

1.4 New Endpoints

Endpoint	Purpose
GET /disclosure-fields?category=	List the required/optional fields for a category — the manufacturer's product form renders itself from this response.
POST /disclosure-fields (platform_admin only)	Define a new field for a category — this is how a second domain (auto parts, FMCG) gets its own requirements without a deploy.
PUT /products/{product_id}/disclosures	Manufacturer submits/updates their values for their category's fields.
GET /products/{product_id}/disclosures	Returns the field definitions and values together — this is what the verify endpoint and any label/export feature pulls from.

1.5 How This Reaches the Consumer and the Regulator

- The public verify response (GET /verify/{unit_id}) gains a disclosures array — populated from the product's category template — shown on the same simple verify page a consumer already sees after scanning the same plain QR.
- This is also what a government inspector needs: scanning the code and having “software applications reveal these data points” is exactly what the regulation itself describes as the compliance target — confirmed by the regulation's own wording, not just this project's interpretation.
- Human-readable label text must still match what's encoded/displayed — the label design (outside this system) should pull the same disclosure values so the printed text and the QR-linked page never disagree.

1.6 Extending the Same Pattern to KYC Document Types (New)

KYC document_type is currently a fixed enum (gst_certificate | industry_license | shop_license | other) — too rigid once an org can hold network edges across multiple industries at once (Section 9).

- Reuse the disclosure_field_definitions pattern for KYC: a kyc_requirements table keyed by category, listing which document types that category expects (e.g. pharma requires a drug licence in addition to GST; auto parts requires a dealer certificate instead).
- An org active in two categories (Section 9) is simply expected to satisfy both categories' requirement rows — no schema change needed when a third category is added later, consistent with the reasoning in Section 1.1.

2. Security & Trust-Model Fixes

2.1 Custody-Transfer Authorization — Currently Not Enforced Synchronously

Confirmed directly in code: the event-processing function validates the state machine transition but never checks whether the scanning org has any real network relationship with the unit's manufacturer before accepting the transition. The unauthorized_route detection rule runs afterward, as a background task — it flags, it does not block.

- Fix: add a synchronous authorization check inside the same request that handles POST /events, before the state transition is applied — resolve the unit's root manufacturer, walk the network_edges graph for an edge covering the

actor org and this product_code, and reject with 403 (UNAUTHORIZED_ROUTE) immediately if none exists, rather than accepting the event and flagging it later.

- Keep the existing async detection for the softer case — an edge exists but is marked unauthorized (distributor_sub tier) — since that traffic is legitimate to record, just lower-trust, and should still flow through and be flagged/scored rather than blocked outright.
- Decide explicitly whether to add an optional expected_recipient_org_id field at DISPATCH time, so a manufacturer can name exactly who a shipment is meant for and only that org's RECEIVE is accepted — tighter than “any authorized org can receive” but a real product decision, not a given; document whichever choice is made.

2.2 Unissued / Never-Shipped Units Currently Verify as “Genuine”

Confirmed directly in code: GET /verify/{unit_id} returns “genuine” for any unit that isn't FLAGGED and isn't recalled — including a unit still sitting in ISSUED state that has never actually been dispatched by the manufacturer. A leaked, never-printed ID would pass verification exactly like a properly sold one.

- Fix: verify should require at least one real custody event (current_state beyond ISSUED, or an explicit “dispatch confirmed” flag) before returning “genuine.” A unit still in ISSUED with zero events should return a distinct result — e.g. not_yet_in_circulation — rather than an unqualified “genuine.”
- This changes the public result set from 3 states to 4: genuine, not_yet_in_circulation, flagged, recalled — update the verify response contract accordingly, and make sure this is the one and only four-state design going forward (distinct from the earlier, now-corrected genuine_unauthorized_channel idea, which this replaces conceptually).

2.3 Consumer Verification Scans Are Invisible to Manufacturers — Decide and Implement

Confirmed directly in code: GET /verify/{unit_id} does not create any Event record. A manufacturer currently has zero visibility into how often or where their products are being checked by real consumers.

- Recommended fix: record an anonymous, aggregate-only verify_count and last_verified_at on the Unit itself (no personal data, no IP, no device ID retained) — gives manufacturers real signal without compromising the no-personal-data-collected principle established earlier in this project.
- Explicitly exclude verify scans from the excessive_scan_frequency detection rule (Section 4 of the Server Production Reference) — a popular product being checked by many genuine customers should never trigger a counterfeit alert.
- New: treat the same verify-count data as a detection signal, not just a manufacturer metric — one retail unit legitimately verified from many distinct geographic locations in a short window (e.g. 10+ cities in 7 days) is itself suspicious, since a single physical item shouldn't be checked from many places. Add a new async detection rule (multi_location_verify) alongside the existing three, raising a low/medium-severity alert rather than blocking anything — this is exactly the kind of signal that turns a previously-invisible data source (Section 2.3) into a genuine anti-counterfeit tool for the one attack this system otherwise cannot see: a copied QR reused by real, unwitting consumers rather than by a registered org.

2.4 Legitimate Route Exceptions — Currently No Mechanism (New)

The network-edge model (Section 2.1) correctly allows any two orgs with a standing authorized relationship to transact freely, but has no path for a genuine one-off need — e.g. a manufacturer needing to emergency-reroute a shipment to an org with no existing network edge at all. Today that scenario has only two bad options: block it entirely, or leave the synchronous check (Section 2.1) loose enough to let it through unchecked for everyone.

- Add a one_time_transfer_exceptions table: exception_id, unit_id or container_id, granted_by_org_id (must be the root manufacturer or an org with authorized standing), recipient_org_id, reason, expires_at.
- Add POST /units/{unit_id}/transfer-exception (or the container equivalent), callable only by an org with authorized standing over that product, creating a short-lived, single-use exception.

- The synchronous authorization check in Section 2.1 checks `network_edges` first, and falls back to checking for a live, unexpired transfer exception before rejecting — so an exception is always an explicit, logged, time-bounded grant, never a silent gap in enforcement.

3. QR & Label Generation — Completing What's Only Partially Built

3.1 Server-Side QR Image Generation — Currently Does Not Exist

Confirmed directly in code: no QR-rendering library is used anywhere in the server. The system only produces a text payload (a URL); turning that into an actual scannable image today depends entirely on the manufacturer's own label software.

- Add QR image rendering (e.g. the `qrcode` Python library) as a new small service, callable per unit or in bulk for a batch, producing PNG or SVG output.
- Error-correction level: use Q ($\approx 25\%$ damage tolerance), not the library default L ($\approx 7\%$) — printed labels get scuffed and partially covered, and a low error-correction level causes genuine products to fail scanning for reasons unrelated to counterfeiting.
- Size configuration: expose a size parameter (physical target size in mm, translated to pixel dimensions at the export DPI) so a manufacturer can generate labels sized correctly for their existing packaging — no fixed one-size-fits-all output.

3.2 Label Export Formats — Only CSV Is Real Today

Confirmed directly in code: `ExportFormat` only defines `csv`; PDF and ZPL are mentioned only in a comment as future work, with no implementation.

- PDF Sheet format: generate a ready-to-print grid of the new QR images (Section 3.1) sized for a specified label sheet — this is the only path available to a manufacturer with no label design software at all, and today it doesn't exist, meaning that manufacturer segment currently has zero way to get printable labels from this system.
- ZPL format: generate raw Zebra printer command output embedding the QR image data — for manufacturers whose thermal printers accept a direct print file without intermediate label software.
- Update the `ExportFormat` enum to include `pdf_sheet` and `zpl` once implemented, and remove the “future releases” comment once they're real.

4. Confirmed Gaps From the Original Audit — Server Side

Gap	Fix
No way to re-list a batch's units after generation	Add GET <code>/batches/{batch_id}/units</code> — confirmed <code>BatchResponse</code> has no <code>unit_ids</code> field and <code>GenerateUnitsResponse</code> only returns them once, at creation.
No MFA for <code>org_admin</code> / <code>platform_admin</code>	Add TOTP enrollment + verification step (e.g. <code>pyotp</code>) in the login flow for these two roles.
No org-rejection endpoint	Add POST <code>/admin/orgs/{org_id}/reject</code> with a required reason string, setting <code>approval_status='rejected'</code> .
No regulator-access list/revoke	Add GET <code>/admin/regulator-access</code> and POST <code>/admin/regulator-access/{id}/revoke</code> .
No session listing for users	Add GET <code>/auth/sessions</code> (the sessions table already has every needed

	field — session_id, device_info, created_at, revoked_at) and POST /auth/sessions/{id}/revoke.
--	---

5. Production Hardening (Restated as Action Items)

- Confirm detection (impossible_travel, unauthorized_route soft-check, excessive_scan_freq) genuinely runs on the background worker (Dockerfile.worker), not inline in the request path — verify directly rather than assume.
- Set explicit SQLAlchemy pool_size and max_overflow sized to (worker processes × pool_size) safely under Postgres's max_connections.
- Confirm CORS allowed_origins is an explicit allowlist in production, never a wildcard.
- Confirm /docs is actually conditionally disabled when settings.is_production is true, not just documented as an intention.
- Add a composite index on events(target_id, timestamp) if not already present.
- Load test POST /events and the container endpoints specifically — the highest-write-volume paths, not just the already-optimized verify endpoint.
- Automated PostgreSQL backups with a tested restore procedure.

6. Target API Surface — All Additions Together

New/Changed Endpoint	Reason
GET /batches/{batch_id}/units	Section 4
GET/POST /disclosure-fields	Section 1
GET/PUT /products/{product_id}/disclosures	Section 1
GET /verify/{unit_id} (response gains disclosures[] and a 4th result state)	Sections 1 & 2.2
POST /batches/{batch_id}/export-labels (format gains pdf_sheet, zpl)	Section 3.2
POST /admin/orgs/{org_id}/reject	Section 4
GET /admin/regulator-access, POST /admin/regulator-access/{id}/revoke	Section 4
GET /auth/sessions, POST /auth/sessions/{id}/revoke	Section 4
MFA enrollment/verification endpoints (exact paths TBD during implementation)	Section 4
POST /units/{unit_id}/transfer-exception	Section 2.4
POST /integrations/webhooks/transporter/{org_id}	Section 9.1
POST /integrations/webhooks/ecommerce/{org_id}	Section 9.2

9. Third-Party Logistics & E-Commerce Integration (New)

ERP integration was already designed (Section 4 of the Manufacturer Integration Guide) and is genuinely covered. Two adjacent integrations were not designed anywhere in this project until now.

9.1 Third-Party Logistics (3PL) Providers

The transporter org_type already exists structurally, but today it only supports a human manually scanning DISPATCH/RECEIVE — a 3PL's own tracking system has no way to submit events automatically.

- Add POST /integrations/webhooks/transporter/{org_id} (authenticated via a per-org webhook secret, not a user session) accepting a pickup/delivery payload from the 3PL's own system and translating it into the same DISPATCH/RECEIVE calls a human would otherwise make.
- This webhook still passes through the same synchronous authorization check (Section 2.1) as any other event — an unauthorized 3PL cannot bypass network-edge enforcement just because the call is machine-to-machine.

9.2 E-Commerce Platforms

Distinct from Social Listings (built for informal individual resellers on social platforms) — this is for a manufacturer's own storefront or a formal marketplace listing, and nothing in this project has designed it until now.

- Add a product_listing_links table connecting a manufacturer's e-commerce SKU (on their own site, Amazon, Flipkart, etc.) to a pool of available unit_ids reserved for that channel.
- Add POST /integrations/webhooks/ecommerce/{org_id} accepting an order-shipped event from the manufacturer's store/marketplace, which automatically consumes one reserved unit_id and submits its DISPATCH — removing the need for a warehouse worker to separately scan every e-commerce order by hand.
- Both webhook integrations should be optional and off by default — a manufacturer using only the mobile/web scanning flow should see no change; this is additive for manufacturers with existing automated fulfillment.

10. Explicitly Out of Scope for This Pass

- Web Dashboard page fixes/builds — covered in the separate Web Dashboard Complete Documentation; this plan is server-only.
- Mobile app — covered in the separate Mobile Build Spec; unaffected by this plan except that it must consume the updated verify response shape (Section 2.2) once built.
- Choosing which second industry vertical to launch — the disclosure system (Section 1) makes this possible later without being a decision this plan needs to make now.

11. Acceptance Checklist

- [] Disclosure field system built; pharma's 9 Schedule H2 fields seeded and functional end-to-end from product form to verify page.
- [] Category-aware KYC document requirements (kyc_requirements table) replacing the fixed document_type enum.
- [] POST /events synchronously rejects an event from an org with no valid network edge — no longer accept-then-flag-later for this case.
- [] One-time transfer exception mechanism implemented, checked as a fallback in the same synchronous authorization path.
- [] A unit in ISSUED state with zero real events no longer returns “genuine” on verify.
- [] Verify scans recorded as an anonymous aggregate count, excluded from excessive-scan-frequency detection, and feeding a new multi_location_verify detection rule.
- [] QR image generation implemented with configurable size and error-correction level Q.
- [] PDF sheet and ZPL export formats implemented and added to ExportFormat.

- [] GET /batches/{batch_id}/units, MFA, org-rejection, regulator-access list/revoke, and session listing all implemented.
- [] Transporter and e-commerce webhook integrations implemented, both off-by-default and passing through the same synchronous authorization check as manual scans.
- [] Production hardening items (Section 5) confirmed or implemented, not just documented as intentions.